MODULE 9: PROJECT PART 4

GitHub Repository Link: https://github.com/devcnx/csd430

styles.css

```
/*
    styles.css - External CSS for Module 4 assignment
    Author: Brittaney Perry-Morgan
    Description: Styles for the JavaBean movie data display.
 */

/* ==========================
   1. GLOBAL STYLES
   ========================== */

*,
html,
body {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: linear-gradient(120deg, #f8fafc 0%, #e0e7ef 100%);
    margin: 0;
    padding: 20px;
    min-height: 100vh;
}

/* ==========================
   2. LAYOUT & CONTAINER
   ========================== */

.container,
.top-title {
    max-width: 900px;
    margin: 0 auto;
    padding: 20px;
}
```

```
.container {
    background: #fff;
    border-radius: 12px;
    box-shadow: 0 4px 24px rgba(0, 0, 0, 0.08);
    margin-bottom: 20px;
}

.top-title {
    text-align: center;
    background: linear-gradient(135deg, #2a3a5e 0%, #1a2540 100%);
    color: #fff;
}

.top-title h1 {
    color: #fff;
    margin: 0 0 10px 0;
}

.top-title h2 {
    color: #e0e7ef;
    margin: 0;
    font-weight: 400;
}

.top-title h2 em {
    font-style: normal;
    color: #a3d9ff;
}

/* ========================
   3. TYPOGRAPHY
   ======================== */

h1,
h2,
h3 {
    color: #2a3a5e;
    margin: 20px 0 15px 0;
    text-transform: uppercase;
}
```

```
h1 {
    font-weight: 200;
    font-size: 1.8rem;
}

h2 {
    font-weight: 400;
    font-size: 1.4rem;
}

h3 {
    font-weight: 600;
    font-size: 1.1rem;
    color: #1a2540;
    border-bottom: 2px solid #e3eaf6;
    padding-bottom: 8px;
}

p.description {
    color: #ffffff;
    line-height: 1.7;
    margin: 15px auto;
    padding: 0 20px;
}

/* ========================
   4. LISTS
   ======================== */

.field-descriptions {
    list-style: none;
    padding: 0 20px;
}

.field-descriptions li {
    padding: 10px 15px;
    margin: 8px 0;
    background: #f1f5fa;
    border-radius: 6px;
    border-left: 4px solid #2a3a5e;
    color: #444;
```

```
}

/* ========================
   5. FORMS
   ========================= */

.form-table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

.form-table tr {
    background: #fff;
}

.form-table td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    vertical-align: top;
}

.form-table td:first-child {
    background: #f9fbfd;
    width: 25%;
    font-weight: 600;
    color: #1a2540;
    vertical-align: middle;
}

.form-table td:last-child {
    width: 75%;
}

.form-table label {
    display: block;
    margin-bottom: 5px;
    font-size: 0.9rem;
    color: #333;
}
```

```
.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    width: 100%;
    padding: 10px 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1rem;
    font-family: 'Segoe UI', Arial, sans-serif;
    box-sizing: border-box;
}

.form-table select {
    background: #fff;
    cursor: pointer;
}

.form-table textarea {
    resize: vertical;
    min-height: 80px;
}

.form-table input[type="checkbox"] {
    width: auto;
    margin-right: 8px;
}

.form-table input.readonly-field {
    background: #e9ecef;
    color: #495057;
    cursor: not-allowed;
}

/* ==========================
   6. NAVIGATION
   ========================== */
```

```
nav ul {
    list-style: none;
    padding: 0 20px;
}

nav ul li {
    margin: 10px 0;
}

nav ul li a {
    color: #2a3a5e;
    text-decoration: none;
    font-weight: 600;
    padding: 8px 12px;
    background: #f1f5fa;
    border-radius: 6px;
    transition: all 0.2s ease;
}

nav ul li a:hover {
    background: #2a3a5e;
    color: #fff;
}

.nav-desc {
    display: block;
    color: #666;
    font-size: 0.85rem;
    margin-top: 8px;
    font-weight: 400;
}

/* ==========================
   6. FORM ACTIONS (for button links)
   ========================== */

.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
```

```
    border-radius: 8px;
    text-align: center;
}

.btn-submit,
.btn-reset {
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
}

/* ========================
   7. FORM ACTIONS (for button links)
   ========================= */
```

```
.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
    border-radius: 8px;
    text-align: center;
}

.btn-submit,
.btn-reset {
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
```

```
}

/* ========================
   8. TABLES
   ========================= */

table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

thead {
    background: #2a3a5e;
    color: #fff;
}

th,
td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    text-align: left;
}

th {
    font-weight: 600;
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

td {
    font-size: 0.95rem;
    color: #333;
    vertical-align: top;
}

tr:nth-child(even) {
    background: #f9fbfd;
}
```

```
tr:hover {
    background: #e8f0ff;
    transition: background 0.2s ease;
}

/* ========================
   9. FOOTER
   ======================== */

footer {
    text-align: center;
    padding: 20px;
    color: #666;
    font-size: 0.9rem;
}

footer p {
    margin: 0;
}

/* ========================
   10. RESPONSIVE DESIGN
   ======================== */

@media (max-width: 768px) {
    body {
        padding: 10px;
    }

    .container,
    .top-title {
        padding: 15px;
    }

    h1 {
        font-size: 1.4rem;
    }

    h2 {
        font-size: 1.1rem;
```

```
}

h3 {
    font-size: 1rem;
}

.form-table {
    margin: 10px 0;
    font-size: 0.85rem;
}

.form-table td {
    padding: 8px 10px;
}

.form-table td:first-child {
    font-size: 0.85rem;
}

.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    font-size: 0.9rem;
    padding: 8px;
}

table {
    margin: 10px 0;
    font-size: 0.85rem;
}

th,
td {
    padding: 8px 10px;
}

p.description {
```

```
    padding: 0 10px;
  }

  .field-descriptions {
    padding: 0 10px;
  }

  .field-descriptions li {
    padding: 8px 12px;
  }

  nav ul {
    padding: 0 10px;
  }

  .form-actions {
    margin: 20px 10px;
    padding: 10px;
  }

  .btn-submit,
  .btn-reset {
    padding: 10px 16px;
    font-size: 0.9rem;
    margin: 5px;
  }
}
```

MovieBean.java

```
/**
 * MovieBean.java
 *
 * CSD430 Server-Side Development
 * Module 9 Project Part 4: CRUD-DELETE Operations
 *
 * Author: Brittaney Perry-Morgan
 *
 * Description: JavaBean for managing movie database operations using JDBC.
 *              Provides methods for database connectivity, movie retrieval,
 *              insertion, updates, and deletion. Used by JSP pages for CRUD operations.
 *
 * Database: CSD430
```

```
 * Table: brittaney_movies_data
 *
 * Fields:
 *   - movie_id (INT, Primary Key, Auto-increment)
 *   - title (VARCHAR)
 *   - genre (VARCHAR)
 *   - release_year (INT)
 *   - rating (VARCHAR)
 *   - director (VARCHAR)
 */

package csd430;

import java.io. *;
import java.sql. *;
import java.util.ArrayList;
import java.util.List;

/**
 * JavaBean class for movie database operations.
 * Handles JDBC connectivity and CRUD operations for the movie database.
 *
 * <p>This bean follows the JavaBeans convention with a no-argument constructor
 * and getter/setter methods. Database credentials are loaded from environment
 * variables or a .env file.</p>
 *
 * <p>Supported operations:</p>
 * <ul>
 *   <li>CREATE - insertMovie()</li>
 *   <li>READ - getAllMovies(), getMovieById(), getAllMovieIds()</li>
 *   <li>UPDATE - updateMovie()</li>
 *   <li>DELETE - deleteMovie(), movieExists()</li>
 * </ul>
 *
 * @author Brittaney Perry-Morgan
 * @version 1.0
 * @since Module 9 Project Part 4
 */
public class MovieBean {

    // Database connection configuration
```

```java
private static String DB_URL;
private static String DB_USER;
private static String DB_PASSWORD;

// Static initialization block - loads database credentials on class load
static {
    loadEnvironmentVariables();
}

/**
 * Loads database connection credentials from environment variables or .env file.
 * Falls back to default values if not found.
 *
 * <p>Environment variables take precedence:
 * <ul>
 *   <li>DB_URL - JDBC connection URL</li>
 *   <li>DB_USER - Database username</li>
 *   <li>DB_PASSWORD - Database password</li>
 * </ul></p>
 *
 * <p>Default fallbacks:
 * <ul>
 *   <li>DB_URL: jdbc:mysql://localhost:3306/CSD430</li>
 *   <li>DB_USER: student1</li>
 *   <li>DB_PASSWORD: pass</li>
 * </ul></p>
 */
private static void loadEnvironmentVariables() {
    DB_URL = System.getenv("DB_URL");
    DB_USER = System.getenv("DB_USER");
    DB_PASSWORD = System.getenv("DB_PASSWORD");

    if (DB_URL == null || DB_USER == null || DB_PASSWORD == null) {
        String envFile = System.getProperty("user.dir") + "/.env";
        if (new File(envFile).exists()) {
            try (BufferedReader reader = new BufferedReader(new FileReader(envFile))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    if (!line.trim().isEmpty() && !line.startsWith("#")) {
                        String[] parts = line.split("=", 2);
                        if (parts.length == 2) {
```

```
                    String key = parts[0].trim();
                    String value = parts[1].trim();
                    switch (key) {
                        case "DB_URL": DB_URL = value; break;
                        case "DB_USER": DB_USER = value; break;
                        case "DB_PASSWORD": DB_PASSWORD = value; break;
                    }
                }
            }
        }
    } catch (IOException e) {
        System.err.println("Error loading .env file: " + e.getMessage());
    }
  }
}

    // Default fallback values
    if (DB_URL == null) DB_URL = "jdbc:mysql://localhost:3306/CSD430";
    if (DB_USER == null) DB_USER = "student1";
    if (DB_PASSWORD == null) DB_PASSWORD = "pass";
}

// Instance variables
private Connection connection;
private List<Integer> movieIds;
private ResultSet resultSet;
private String errorMessage;

/**
 * Default constructor - initializes the movie ID list and error message.
 * Required for JavaBeans compatibility.
 */
public MovieBean() {
    movieIds = new ArrayList<Integer>();
    errorMessage = null;
}

/**
 * Establishes a connection to the MySQL database using loaded credentials.
 * Loads the MySQL JDBC driver and creates a connection.
 *
```

```
 * @return true if connection successful, false otherwise
 */
public boolean connect() {
    try {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            Class.forName("com.mysql.jdbc.Driver");
        }
        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        return true;
    } catch (ClassNotFoundException e) {
        errorMessage = "MySQL Driver not found: " + e.getMessage();
        return false;
    } catch (SQLException e) {
        errorMessage = "Database connection error: " + e.getMessage();
        return false;
    }
}

/**
 * Closes the database connection and any open result sets.
 * Should be called when database operations are complete.
 */
public void disconnect() {
    try {
        if (resultSet != null) {
            resultSet.close();
        }
        if (connection != null) {
            connection.close();
        }
    } catch (SQLException e) {
        errorMessage = "Error closing connection: " + e.getMessage();
    }
}

/**
 * Retrieves all movie IDs from the database and stores them in the movieIds list.
 * Results are ordered by movie_id.
 *
```

```
 * @return true if successful, false if an error occurred
 */
public boolean getAllMovieIds() {
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT movie_id FROM brittaney_movies_data ORDER BY movie_id";
        PreparedStatement stmt = connection.prepareStatement(query);
        resultSet = stmt.executeQuery();

        movieIds.clear();
        while (resultSet.next()) {
            movieIds.add(resultSet.getInt("movie_id"));
        }
        resultSet.close();
        stmt.close();
        return true;
    } catch (SQLException e) {
        errorMessage = "Error fetching movie IDs: " + e.getMessage();
        return false;
    }
}

/**
 * Retrieves a single movie by its ID and populates the result set.
 * Used for fetching movie details for editing or deletion confirmation.
 *
 * @param movieId the primary key of the movie to retrieve
 * @return true if movie found, false if not found or error occurred
 */
public boolean getMovieById(int movieId) {
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }
```

```
    try {
        String query = "SELECT * FROM brittaney_movies_data WHERE movie_id = ?";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setInt(1, movieId);
        resultSet = stmt.executeQuery();

        if (!resultSet.next()) {
            errorMessage = "Movie not found";
            resultSet.close();
            stmt.close();
            return false;
        }
        return true;
    } catch (SQLException e) {
        errorMessage = "Error fetching movie: " + e.getMessage();
        return false;
    }
}

/**
 * Retrieves all movies from the database in a result set.
 * Results are ordered by movie_id.
 *
 * @return true if successful, false if an error occurred
 */
public boolean getAllMovies() {
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT * FROM brittaney_movies_data ORDER BY movie_id";
        PreparedStatement stmt = connection.prepareStatement(query);
        resultSet = stmt.executeQuery();
        return true;
    } catch (SQLException e) {
        errorMessage = "Error fetching movies: " + e.getMessage();
        return false;
    }
```

```
}

/**
 * Inserts a new movie record into the database.
 * The movie_id is auto-generated by the database.
 *
 * @param title the movie title
 * @param genre the movie genre
 * @param releaseYear the year of release
 * @param rating the MPAA rating
 * @param director the director's name
 * @return true if insert successful, false otherwise
 */
public boolean insertMovie(String title, String genre, int releaseYear, String rating, String director) {
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "INSERT INTO brittaney_movies_data (title, genre, release_year, rating, director) VALUES (?, ?, ?, ?, ?)";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setString(1, title);
        stmt.setString(2, genre);
        stmt.setInt(3, releaseYear);
        stmt.setString(4, rating);
        stmt.setString(5, director);

        int rowsAffected = stmt.executeUpdate();
        stmt.close();
        return rowsAffected > 0;
    } catch (SQLException e) {
        errorMessage = "Error inserting movie: " + e.getMessage();
        return false;
    }
}

/**
 * Updates an existing movie record in the database.
 * All fields except movie_id can be modified.
```

```
     *
     * @param movieId the primary key of the movie to update
     * @param title the new movie title
     * @param genre the new movie genre
     * @param releaseYear the new release year
     * @param rating the new MPAA rating
     * @param director the new director's name
     * @return true if update successful, false otherwise
     */
    public boolean updateMovie(int movieId, String title, String genre, int releaseYear, String rating, String director) {
        if (connection == null) {
            if (!connect()) {
                return false;
            }
        }

        try {
            String query = "UPDATE brittaney_movies_data SET title = ?, genre = ?, release_year = ?, rating = ?, director = ? WHERE
movie_id = ?";
            PreparedStatement stmt = connection.prepareStatement(query);
            stmt.setString(1, title);
            stmt.setString(2, genre);
            stmt.setInt(3, releaseYear);
            stmt.setString(4, rating);
            stmt.setString(5, director);
            stmt.setInt(6, movieId);

            int rowsAffected = stmt.executeUpdate();
            stmt.close();
            return rowsAffected > 0;
        } catch (SQLException e) {
            errorMessage = "Error updating movie: " + e.getMessage();
            return false;
        }
    }

    /**
     * Deletes a movie record from the database.
     * Module 9 - DELETE Operation
     *
     * @param movieId the primary key of the movie to delete
```

```
 * @return true if deletion successful, false otherwise
 */
public boolean deleteMovie(int movieId) {
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "DELETE FROM brittaney_movies_data WHERE movie_id = ?";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setInt(1, movieId);

        int rowsAffected = stmt.executeUpdate();
        stmt.close();

        if (rowsAffected > 0) {
            System.out.println("Movie deleted successfully: ID " + movieId);
            return true;
        } else {
            errorMessage = "No movie found with ID: " + movieId;
            return false;
        }
    } catch (SQLException e) {
        errorMessage = "Error deleting movie: " + e.getMessage();
        return false;
    }
}

/**
 * Checks if a movie exists in the database.
 * Module 9 - DELETE Operation
 * Used to verify existence before deletion.
 *
 * @param movieId the movie ID to check
 * @return true if movie exists, false otherwise
 */
public boolean movieExists(int movieId) {
    if (connection == null) {
        if (!connect()) {
```

```
            return false;
        }
    }

    try {
        String query = "SELECT COUNT(*) FROM brittaney_movies_data WHERE movie_id = ?";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setInt(1, movieId);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            boolean exists = rs.getInt(1) > 0;
            rs.close();
            stmt.close();
            return exists;
        }
        rs.close();
        stmt.close();
        return false;
    } catch (SQLException e) {
        errorMessage = "Error checking movie existence: " + e.getMessage();
        return false;
    }
}

/**
 * Returns the list of movie IDs from the last getAllMovieIds() call.
 *
 * @return List of movie IDs
 */
public List<Integer> getMovieIds() {
    return movieIds;
}

/**
 * Returns the movie_id from the current result set position.
 *
 * @return the movie ID, or 0 if error
 */
public int getMovieId() {
    try {
```

```
        return resultSet.getInt("movie_id");
    } catch (SQLException e) {
        return 0;
    }
}

/**
 * Returns the title from the current result set position.
 *
 * @return the movie title, or empty string if error
 */
public String getTitle() {
    try {
        return resultSet.getString("title");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the genre from the current result set position.
 *
 * @return the movie genre, or empty string if error
 */
public String getGenre() {
    try {
        return resultSet.getString("genre");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the release year from the current result set position.
 *
 * @return the release year, or 0 if error
 */
public int getReleaseYear() {
    try {
        return resultSet.getInt("release_year");
    } catch (SQLException e) {
```

```
        return 0;
    }
}

/**
 * Returns the rating from the current result set position.
 *
 * @return the MPAA rating, or empty string if error
 */
public String getRating() {
    try {
        return resultSet.getString("rating");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the director from the current result set position.
 *
 * @return the director name, or empty string if error
 */
public String getDirector() {
    try {
        return resultSet.getString("director");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the last error message from database operations.
 *
 * @return error message string, or null if no error
 */
public String getErrorMessage() {
    return errorMessage;
}

/**
 * Advances the result set to the next row.
```

```
     * Used for iterating through query results.
     *
     * @return true if there is a next row, false if end of results
     */
    public boolean next() {
        try {
            return resultSet.next();
        } catch (SQLException e) {
            errorMessage = "Error navigating result set: " + e.getMessage();
            return false;
        }
    }
}
```

index.jsp

```
<%--
    CSD430 Server-Side Development

    Module 9 Project Part 4: CRUD-DELETE Operations
    Author: Brittaney Perry-Morgan

    Description: Course index page with navigation to movie database
    application demonstrating JDBC connectivity and JavaBeans for
    CRUD-DELETE operations.

    index.jsp - Course Index Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Module 9 Project Part 4 - CRUD-DELETE Operations</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 9 Project Part 4</h1>
        <h2>JDBC & JavaBeans - <em>CRUD-DELETE Operations</em></h2>
        <p class="description">Author: Brittaney Perry-Morgan</p>
    </div>
```

```
<div class="container">
    <h3>Movie Database CRUD Operations</h3>
    <p class="description">
        This web application demonstrates database connectivity using JDBC
        and JavaBeans for Create, Read, Update, and Delete (CRUD) operations.
        Module 9 focuses on <strong>DELETE</strong> operations - removing
        movie records from the database.
    </p>

    <h3>Database Information</h3>
    <ul class="field-descriptions">
        <li><strong>Database Name:</strong> CSD430</li>
        <li><strong>Table Name:</strong> brittaney_movies_data</li>
        <li><strong>Username:</strong> student1</li>
        <li><strong>Topic:</strong> Movies</li>
        <li><strong>Fields:</strong> 6 (movie_id, title, genre, release_year, rating, director)</li>
    </ul>

    <h3>Navigation - DELETE Operations</h3>
    <nav>
        <ul>
            <li>
                <a href="movieDelete.jsp">Delete Movie</a>
                <span class="nav-desc">Select a movie to delete from the database</span>
            </li>
            <li>
                <a href="/module_05_and_06_project_1/movieSelection.jsp">View Movies (Module 5&6)</a>
                <span class="nav-desc">View all movie records (READ operation)</span>
            </li>
            <li>
                <a href="/module_07_project_2/movieForm.jsp">Add Movie (Module 7)</a>
                <span class="nav-desc">Create a new movie record (CREATE operation)</span>
            </li>
            <li>
                <a href="/module_08_project_3/movieSelection.jsp">Edit Movie (Module 8)</a>
                <span class="nav-desc">Update existing movie records (UPDATE operation)</span>
            </li>
        </ul>
    </nav>
```

```
        <h3>DELETE Operation Flow</h3>
        <ul class="field-descriptions">
            <li><strong>Step 1:</strong> View all movie records in a table format</li>
            <li><strong>Step 2:</strong> Select a movie ID from the dropdown menu</li>
            <li><strong>Step 3:</strong> Review movie details and confirm deletion</li>
            <li><strong>Step 4:</strong> View remaining records after deletion</li>
            <li><strong>Step 5:</strong> Optionally select another movie to delete</li>
        </ul>
    </div>

    <footer>
        <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
    </footer>
</body>
</html>
```

movieConfirmDelete.jsp

```
<%--
    CSD430 Server-Side Development

    Module 9 Project Part 4: CRUD-DELETE Operations
    Author: Brittaney Perry-Morgan

    Description: Confirmation page displaying movie details before deletion.
    Shows all field values and requires user confirmation.

    movieConfirmDelete.jsp - Delete Confirmation Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Confirm Delete - CSD430</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 9 Project Part 4</h1>
        <h2>Movie Database - <em>DELETE Operation</em></h2>
```

```
        <p class="description">Confirm movie deletion.</p>
    </div>

    <div class="container">
<%
        MovieBean movieBean = new MovieBean();
        String movieIdParam = request.getParameter("movieId");

        if (movieIdParam == null || movieIdParam.isEmpty()) {
%>
        <div class="error-message">
            <p>No movie selected. Please go back and select a movie to delete.</p>
        </div>
        <div class="form-actions">
            <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
            <a href="index.jsp" class="btn-reset">Back to Course Index</a>
        </div>
<%
        } else {
            try {
                int movieId = Integer.parseInt(movieIdParam);
                boolean connected = movieBean.connect();

                if (!connected) {
%>
        <div class="error-message">
            <p>Database Connection Error:</p>
            <p><%= movieBean.getErrorMessage() %></p>
        </div>
        <div class="form-actions">
            <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
            <a href="index.jsp" class="btn-reset">Back to Course Index</a>
        </div>
<%
                } else {
                    boolean success = movieBean.getMovieById(movieId);

                    if (!success) {
%>
        <div class="error-message">
            <p>Error retrieving movie data:</p>
```

```
        <p><%= movieBean.getErrorMessage() %></p>
    </div>
    <div class="form-actions">
        <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
<%
            } else {
                // Store values before disconnecting
                String title = movieBean.getTitle();
                String genre = movieBean.getGenre();
                int releaseYear = movieBean.getReleaseYear();
                String rating = movieBean.getRating();
                String director = movieBean.getDirector();
                movieBean.disconnect();
%>

    <div class="warning-message">
        <p><strong>&#9888; Warning:</strong> You are about to permanently delete this movie record. This action cannot be
undone!</p>
    </div>

    <h3>Movie to Delete</h3>
    <p class="description">
        Please review the movie details below before confirming deletion.
    </p>

    <table>
        <thead>
            <tr>
                <th>Field</th>
                <th>Value</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td><strong>Movie ID</strong></td>
                <td><%= movieId %></td>
            </tr>
            <tr>
                <td><strong>Title</strong></td>
```

```
                <td><%= title %></td>
            </tr>
            <tr>
                <td><strong>Genre</strong></td>
                <td><%= genre %></td>
            </tr>
            <tr>
                <td><strong>Release Year</strong></td>
                <td><%= releaseYear %></td>
            </tr>
            <tr>
                <td><strong>Rating</strong></td>
                <td><%= rating %></td>
            </tr>
            <tr>
                <td><strong>Director</strong></td>
                <td><%= director %></td>
            </tr>
        </tbody>
    </table>

    <h3>Field Descriptions</h3>
    <ul class="field-descriptions">
        <li><strong>Movie ID:</strong> Unique identifier (Primary Key, will be permanently removed)</li>
        <li><strong>Title:</strong> The name of the movie</li>
        <li><strong>Genre:</strong> The category of the movie</li>
        <li><strong>Release Year:</strong> Year the movie was released</li>
        <li><strong>Rating:</strong> MPAA rating</li>
        <li><strong>Director:</strong> Director of the movie</li>
    </ul>

    <div class="form-actions">
        <a href="movieProcessDelete.jsp?movieId=<%= movieId %>" class="btn-submit" style="background-color: #dc3545;">Yes,
Delete Movie</a>
        <a href="movieDelete.jsp" class="btn-reset">Cancel</a>
    </div>

    <div class="form-actions">
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
<%
```

```
                }
            }
        } catch (NumberFormatException e) {
%>
    <div class="error-message">
        <p>Invalid movie ID: <%= movieIdParam %></p>
    </div>
    <div class="form-actions">
        <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
<%
        }
    }
%>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
</body>
</html>
```

movieProcessDelete.jsp

```
<%--
  CSD430 Server-Side Development

  Module 9 Project Part 4: CRUD-DELETE Operations
  Author: Brittaney Perry-Morgan

  Description: Processes the movie deletion and displays the remaining records
  in a table format with a dropdown for selecting another movie to delete.

  movieProcessDelete.jsp - Delete Processing & Results Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<%@ page import="java.util.List" %>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Delete Result - CSD430</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 9 Project Part 4</h1>
        <h2>Movie Database - <em>DELETE Operation</em></h2>
        <p class="description">Delete operation result and remaining records.</p>
    </div>

    <div class="container">
<%
        String movieIdParam = request.getParameter("movieId");

        if (movieIdParam == null || movieIdParam.isEmpty()) {
%>
        <div class="error-message">
            <p>No movie ID provided. Please select a movie to delete.</p>
        </div>
        <div class="form-actions">
            <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
            <a href="index.jsp" class="btn-reset">Back to Course Index</a>
        </div>
<%
        } else {
            try {
                int movieId = Integer.parseInt(movieIdParam);

                MovieBean movieBean = new MovieBean();
                boolean connected = movieBean.connect();

                if (!connected) {
%>
        <div class="error-message">
            <p>Database Connection Error:</p>
            <p><%= movieBean.getErrorMessage() %></p>
        </div>
        <div class="form-actions">
            <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
            <a href="index.jsp" class="btn-reset">Back to Course Index</a>
```

```
    </div>
<%
        } else {
            // Check if movie exists
            if (!movieBean.movieExists(movieId)) {
%>
    <div class="error-message">
      <p>Movie not found with ID: <%= movieId %></p>
      <p>The movie may have already been deleted.</p>
    </div>
<%
            } else {
                // Delete the movie
                boolean success = movieBean.deleteMovie(movieId);

                if (success) {
%>
    <div class="success-message">
      <p>Movie with ID <%= movieId %> deleted successfully!</p>
    </div>
<%
                } else {
%>
    <div class="error-message">
      <p>Failed to delete movie: <%= movieBean.getErrorMessage() %></p>
    </div>
<%
                }
            }

            // Display remaining records
            movieBean.disconnect();
            movieBean.connect();
            boolean gotMovies = movieBean.getAllMovies();
%>

    <h3>Remaining Movie Records</h3>
    <p class="description">
      The table below displays all movie records currently stored in the database
      after the deletion operation.
    </p>
```

```
    <table>
        <thead>
            <tr>
                <th>Movie ID</th>
                <th>Title</th>
                <th>Genre</th>
                <th>Release Year</th>
                <th>Rating</th>
                <th>Director</th>
            </tr>
        </thead>
        <tbody>
<%
                int recordCount = 0;
                while (movieBean.next()) {
                    recordCount++;
%>
            <tr>
                <td><%= movieBean.getMovieId() %></td>
                <td><%= movieBean.getTitle() %></td>
                <td><%= movieBean.getGenre() %></td>
                <td><%= movieBean.getReleaseYear() %></td>
                <td><%= movieBean.getRating() %></td>
                <td><%= movieBean.getDirector() %></td>
            </tr>
<%
                }
                movieBean.disconnect();
%>
        </tbody>
    </table>

    <p class="record-count">Total Records: <%= recordCount %></p>

<%
                if (recordCount == 0) {
%>
    <div class="warning-message">
        <p><strong>Empty Table:</strong> All movie records have been deleted from the database.</p>
        <p>The table structure (thead row) remains, but there are no data rows to display.</p>
```

```
    </div>
    <div class="form-actions">
        <a href="/module_07_project_2/movieForm.jsp" class="btn-submit">Add New Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
<%
            } else {
%>

    <h3>Delete Another Movie</h3>
    <p class="description">
        Select another movie from the dropdown menu to delete additional records.
    </p>

    <form action="movieConfirmDelete.jsp" method="get">
        <table class="form-table">
            <tr>
                <td><label for="movieId">Select Movie ID:</label></td>
                <td>
                    <select id="movieId" name="movieId" required>
                        <option value="">-- Select a Movie --</option>
<%
                    // Reconnect and get remaining IDs
                    movieBean.connect();
                    movieBean.getAllMovies();
                    while (movieBean.next()) {
%>
                        <option value="<%= movieBean.getMovieId() %>">
                            <%= movieBean.getMovieId() %> - <%= movieBean.getTitle() %>
                        </option>
<%
                    }
                    movieBean.disconnect();
%>
                    </select>
                </td>
            </tr>
        </table>

        <div class="form-actions">
            <button type="submit" class="btn-submit" style="background-color: #dc3545;">Delete Selected Movie</button>
```

```
            <a href="index.jsp" class="btn-reset">Back to Course Index</a>
        </div>
    </form>

    <h3>Field Descriptions</h3>
    <ul class="field-descriptions">
        <li><strong>Movie ID:</strong> Unique identifier (Primary Key)</li>
        <li><strong>Title:</strong> The name of the movie</li>
        <li><strong>Genre:</strong> The category of the movie</li>
        <li><strong>Release Year:</strong> Year the movie was released</li>
        <li><strong>Rating:</strong> MPAA rating</li>
        <li><strong>Director:</strong> Director of the movie</li>
    </ul>

    <div class="form-actions">
        <a href="/module_07_project_2/movieForm.jsp" class="btn-reset">Add New Movie (Module 7)</a>
        <a href="/module_08_project_3/movieSelection.jsp" class="btn-reset">Edit Movie (Module 8)</a>
    </div>
<%
                }
            }
        } catch (NumberFormatException e) {
%>
    <div class="error-message">
        <p>Invalid movie ID: <%= movieIdParam %></p>
    </div>
    <div class="form-actions">
        <a href="movieDelete.jsp" class="btn-submit">Select Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
<%
        }
    }
%>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
</body>
</html>
```

deliverables

MODULE 9 PROJECT PART 4

JDBC & JAVABEANS - CRUD-DELETE OPERATIONS

Author: Brittaney Perry-Morgan

MOVIE DATABASE CRUD OPERATIONS

DATABASE INFORMATION

- **Database Name:** CSD430
- **Table Name:** brittaney_movies_data
- **Username:** student1
- **Topic:** Movies
- **Fields:** 6 (movie_id, title, genre, release_year, rating, director)

NAVIGATION - DELETE OPERATIONS

Delete Movie
Select a movie to delete from the database

View Movies (Module 5&6)
View all movie records (READ operation)

Add Movie (Module 7)
Create a new movie record (CREATE operation)

Edit Movie (Module 8)
Update existing movie records (UPDATE operation)

DELETE OPERATION FLOW

- **Step 1:** View all movie records in a table format
- **Step 2:** Select a movie ID from the dropdown menu
- **Step 3:** Review movie details and confirm deletion
- **Step 4:** View remaining records after deletion
- **Step 5:** Optionally select another movie to delete

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 9 PROJECT PART 4

MOVIE DATABASE - DELETE OPERATION

Select a movie record to delete from the database.

ALL MOVIE RECORDS

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
1	The Shawshank Redemption	Action	1994	R	Frank Darabont
2	The Godfather	Crime	1972	R	Francis Ford Coppola
3	The Dark Knight	Action	2008	PG-13	Christopher Nolan
4	Pulp Fiction	Crime	1994	R	Quentin Tarantino
5	The Lord of the Rings: The Return of the King	Fantasy	2003	PG-13	Peter Jackson
6	Forrest Gump	Drama	1994	PG-13	Robert Zemeckis
7	Inception	Sci-Fi	2010	PG-13	Christopher Nolan
8	The Matrix	Sci-Fi	1999	R	The Wachowskis
9	Titanic	Romance	1997	PG-13	James Cameron
10	The Silence of the Lambs	Thriller	1991	R	Jonathan Demme

Total Records: 10

DELETE MOVIE

Select Movie ID: -- Select a Movie --

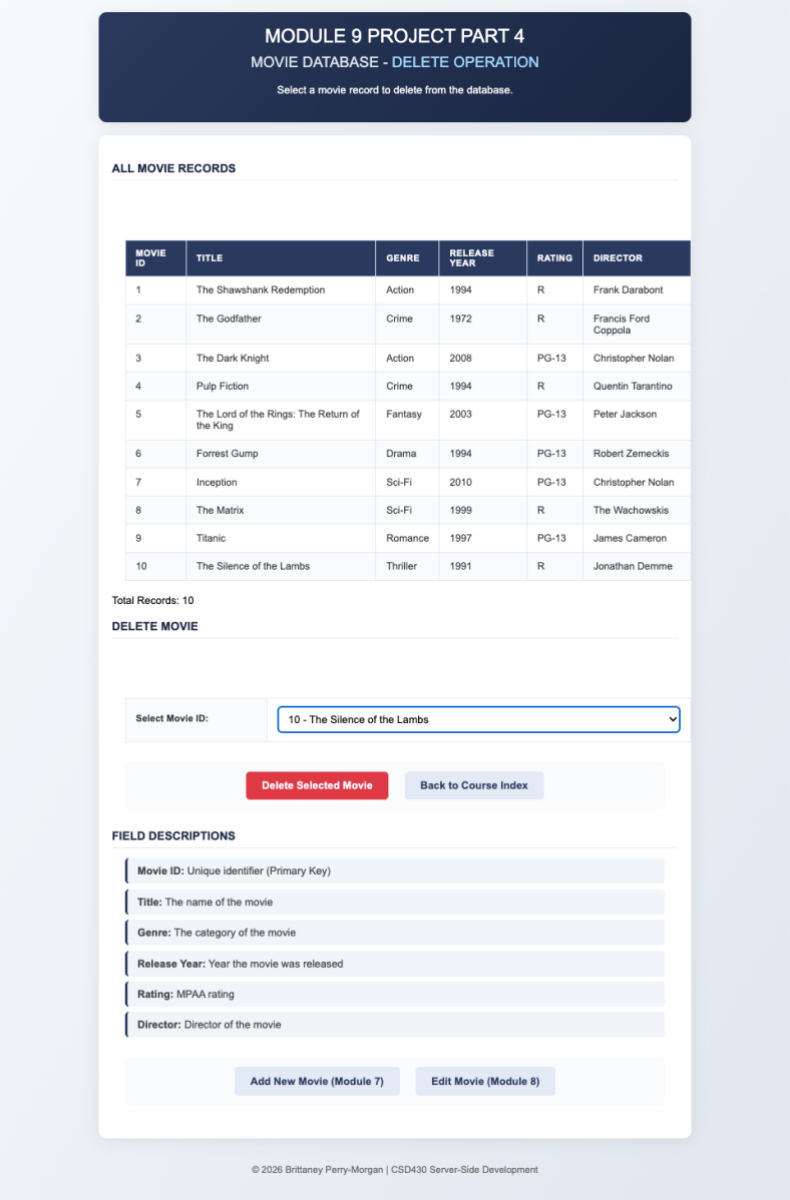
Delete Selected Movie | Back to Course Index

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier (Primary Key)
- **Title:** The name of the movie
- **Genre:** The category of the movie
- **Release Year:** Year the movie was released
- **Rating:** MPAA rating
- **Director:** Director of the movie

Add New Movie (Module 7) | Edit Movie (Module 8)

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development



MODULE 9 PROJECT PART 4

MOVIE DATABASE - DELETE OPERATION

Select a movie record to delete from the database.

ALL MOVIE RECORDS

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
1	The Shawshank Redemption	Action	1994	R	Frank Darabont
2	The Godfather	Crime	1972	R	Francis Ford Coppola
3	The Dark Knight	Action	2008	PG-13	Christopher Nolan
4	Pulp Fiction	Crime	1994	R	Quentin Tarantino
5	The Lord of the Rings: The Return of the King	Fantasy	2003	PG-13	Peter Jackson
6	Forrest Gump	Drama	1994	PG-13	Robert Zemeckis
7	Inception	Sci-Fi	2010	PG-13	Christopher Nolan
8	The Matrix	Sci-Fi	1999	R	The Wachowskis
9	Titanic	Romance	1997	PG-13	James Cameron
10	The Silence of the Lambs	Thriller	1991	R	Jonathan Demme

Total Records: 10

DELETE MOVIE

Select Movie ID: 10 - The Silence of the Lambs

Delete Selected Movie | Back to Course Index

FIELD DESCRIPTIONS

Movie ID: Unique identifier (Primary Key)

Title: The name of the movie

Genre: The category of the movie

Release Year: Year the movie was released

Rating: MPAA rating

Director: Director of the movie

Add New Movie (Module 7) | Edit Movie (Module 8)

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 9 PROJECT PART 4

MOVIE DATABASE - DELETE OPERATION

Confirm movie deletion.

⚠ **Warning:** You are about to permanently delete this movie record. This action cannot be undone!

MOVIE TO DELETE

FIELD	VALUE
Movie ID	10
Title	The Silence of the Lambs
Genre	Thriller
Release Year	1991
Rating	R
Director	Jonathan Demme

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier (Primary Key, will be permanently removed)
- **Title:** The name of the movie
- **Genre:** The category of the movie
- **Release Year:** Year the movie was released
- **Rating:** MPAA rating
- **Director:** Director of the movie

Yes, Delete Movie | Cancel

Back to Course Index

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 9 PROJECT PART 4

MOVIE DATABASE - DELETE OPERATION

Delete operation result and remaining records.

Movie with ID 10 deleted successfully!

REMAINING MOVIE RECORDS

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
1	The Shawshank Redemption	Action	1994	R	Frank Darabont
2	The Godfather	Crime	1972	R	Francis Ford Coppola
3	The Dark Knight	Action	2008	PG-13	Christopher Nolan
4	Pulp Fiction	Crime	1994	R	Quentin Tarantino
5	The Lord of the Rings: The Return of the King	Fantasy	2003	PG-13	Peter Jackson
6	Forrest Gump	Drama	1994	PG-13	Robert Zemeckis
7	Inception	Sci-Fi	2010	PG-13	Christopher Nolan
8	The Matrix	Sci-Fi	1999	R	The Wachowskis
9	Titanic	Romance	1997	PG-13	James Cameron

Total Records: 9

DELETE ANOTHER MOVIE

Select Movie ID: -- Select a Movie --

Delete Selected Movie | Back to Course Index

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier (Primary Key)
- **Title:** The name of the movie
- **Genre:** The category of the movie
- **Release Year:** Year the movie was released
- **Rating:** MPAA rating
- **Director:** Director of the movie

Add New Movie (Module 7) | Edit Movie (Module 8)

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development

MODULE 9 PROJECT PART 4

MOVIE DATABASE - DELETE OPERATION

Delete operation result and remaining records.

Movie not found with ID: 10
The movie may have already been deleted.

REMAINING MOVIE RECORDS

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
2	The Godfather	Crime	1972	R	Francis Ford Coppola
3	The Dark Knight	Action	2008	PG-13	Christopher Nolan
4	Pulp Fiction	Crime	1994	R	Quentin Tarantino
5	The Lord of the Rings: The Return of the King	Fantasy	2003	PG-13	Peter Jackson
6	Forrest Gump	Drama	1994	PG-13	Robert Zemeckis
7	Inception	Sci-Fi	2010	PG-13	Christopher Nolan
8	The Matrix	Sci-Fi	1999	R	The Wachowskis
9	Titanic	Romance	1997	PG-13	James Cameron

Total Records: 8

DELETE ANOTHER MOVIE

Select Movie ID: -- Select a Movie --

Delete Selected Movie | **Back to Course Index**

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier (Primary Key)
- **Title:** The name of the movie
- **Genre:** The category of the movie
- **Release Year:** Year the movie was released
- **Rating:** MPAA rating
- **Director:** Director of the movie

Add New Movie (Module 7) | **Edit Movie (Module 8)**

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development